



BLOCK 2

BACKEND PYTHON FROM SCRATCH

Manel Ocana

INDEX

- Introduction
- Choix techniques
- Accès au serveur
- Approche de la présentation
- Technologies utilisées
- Architecture du projet (MCV)
- Controllers (logique métier)
- Models (accès a base de données)
- Response Helper
- Fonctionnement global
- Endpoints principaux
- Sécurité et gestion des rôles
- Base de données
- Tests et validation
- Conclusion

INTRODUCTION

Ce projet consiste en la création d'une API backend complète développée en Python pur, sans utiliser de framework.

L'objectif principal est de comprendre en profondeur le fonctionnement d'un backend, notamment la gestion des requêtes HTTP, la logique métier, et la communication avec une base de données.

Ce projet permet de reproduire le fonctionnement interne d'un framework, afin de mieux comprendre ce qui se passe "derrière la magie".

CHOIX TECHNIQUES

- comprendre le fonctionnement interne d'un backend
- maîtriser les requêtes HTTP à bas niveau
- apprendre la structure MVC sans abstraction
- renforcer la logique backend pure

ACCÈS AU SERVEUR

- Le backend fonctionne en local sur un serveur HTTP simple basé sur `http.server`.
- Il peut être lancé avec la commande :
- `python main.py`
- Ensuite, l'API est accessible via :
- `http://localhost:8000`
- Les tests des routes ont été réalisés avec Thunder Client, afin de simuler des requêtes

HTTP (GET, POST, PUT, DELETE).

APPROCHE DE LA PRÉSENTATION

Dans cette présentation, je vais expliquer le fonctionnement global de l'API ainsi que l'architecture mise en place.

Je ne vais pas entrer dans tous les détails du code, mais l'ensemble du projet est disponible sur GitHub pour consultation.

https://github.com/manelocana/Projet_block_2_backend/blob/main/README.md

TECHNOLOGIES UTILISÉES

Pour ce projet, j'ai utilisé :

- Python (backend sans framework)
- http.server (gestion des requêtes HTTP)
- MySQL / SQLite (database selon configuration)
- JSON (échange de données entre client et serveur)
- Thunder Client (tests des endpoints)

ARCHITECTURE DU PROJET (MCV)

- Le projet est structuré selon le modèle MVC (Model - View - Controller), permettant une séparation claire des responsabilités.

Handler (point d'entrée principal de la app)

Rôle :

- Gérer les requêtes HTTP entrantes
- Identifier la route demandée
- Rediriger vers le contrôleur approprié
- Traiter les méthodes HTTP : GET, POST, PUT, DELETE

En résumé : il agit comme un routeur central

CONTROLLERS (LOGIQUE MÉTIER)

- Les Controllers contiennent la logique de l'application.

Rôle :

- Recevoir les données depuis le Handler
- Valider les données (règles métier)
- Appeler les modèles pour accéder à la base de données
- Retourner une réponse (souvent en JSON)

Exemples :

- UserController
- ArtworkController
- BiographyController
- MessageController

En résumé : ils prennent les décisions

MODELS (ACCÈS A BASE DE DONNÉES)

Les Models gèrent l'interaction avec la base de données.
Sont les tables de mysql.

Rôle :

- Exécuter les requêtes SQL
- Encapsuler la logique d'accès aux données
- Séparer la base de données du reste de l'application

Exemples :

- User
- Artwork
- Biography
- Message

RESPONSE HELPER

Une classe Response a été créée pour standardiser les réponses HTTP.

Fonctionnalités :

- Conversion des données en JSON
- Ajout des headers HTTP
- Gestion des codes de statut (200, 404, 500, etc.)

Elle uniformise les réponses de l'API

FONCTIONNEMENT GLOBAL

Le flux de l'application est le suivant :

- Le client envoie une requête HTTP
- Le Handler analyse la route et la méthode
- Le Controller traite la logique métier
- Le Model interagit avec la base de données
- Une réponse JSON est renvoyée au client

ENDPOINTS PRINCIPAUX

L'API expose plusieurs routes :

Authentication

- POST /api/register
- POST /api/login

Artworks

- GET /api/artworks
- POST /api/artworks
- PUT /api/artworks/:id
- DELETE /api/artworks/:id

Utilisateurs

- GET /api/users (protégé admin)

Biography

- GET /api/biography
- PUT /api/biography

Messages

- POST /api/contact
- GET /api/messages (admin)

SÉCURITÉ ET GESTION DES RÔLES

Une gestion simple des rôles a été implémentée via les headers HTTP :

- Role: admin pour les routes protégées
- User-Id pour identifier l'utilisateur connecté

Cela permet de simuler un système d'autorisation basique.

BASE DE DONNÉES

```
manel@debian: ~  
mysql> DESCRIBE artworks;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| id | int | NO | PRI | NULL | auto |  
_increment |  
| user_id | int | NO | MUL | NULL | |  
| title | varchar(255) | NO | | NULL | |  
| description | text | YES | | NULL | |  
| category | varchar(100) | YES | | NULL | |  
| created_at | timestamp | YES | | CURRENT_TIMESTAMP | DEFA  
ULT_GENERATED |  
+-----+-----+-----+-----+-----+-----+
```

La base de données est relationnelle et contient plusieurs tables :

- users
- artworks
- biography
- messages

Chaque modèle contient des méthodes pour :

- créer des données
- lire des données
- modifier des données
- supprimer des données

TESTS ET VALIDATION

Les routes ont été testées avec Thunder Client afin de vérifier :

- le bon fonctionnement des endpoints
- la gestion des erreurs
- les réponses JSON
- les statuts HTTP

```
""" verifier que email c'est unique """  
def unique_email():  
    return f"test_{int(time.time() * 1000)}@test.com"
```

VS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
) manel@debian:~/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_block_2
```

```
) manel@debian:~/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_block_2
```

```
===== test session starts =====  
platform linux -- Python 3.11.2, pytest-9.0.2, pluggy-1.6.0 -- /home/manel/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_block_2
```

```
dir: .pytest_cache
```

```
dir: /home/manel/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_block_2
```

```
collected 13 items
```

```
/test_artwork_controller.py::test_create_artwork_success PASSED
```

```
/test_artwork_controller.py::test_get_all_artworks PASSED
```

```
/test_artwork_controller.py::test_create_artwork_missing_title PASSED
```

```
/test_artwork_controller.py::test_update_artwork PASSED
```

```
/test_artwork_controller.py::test_delete_artwork PASSED
```

```
/test_biography.py::test_update_biography PASSED
```

```
/test_user_controller.py::test_register_success PASSED
```

```
/test_user_controller.py::test_register_missing_fields PASSED
```

```
/test_user_controller.py::test_register_duplicate_email PASSED
```

```
/test_user_controller.py::test_login_success PASSED
```

```
/test_user_controller.py::test_login_wrong_password PASSED
```

```
/test_user_controller.py::test_login_user_not_found PASSED
```

```
/test_user_controller.py::test_update_user PASSED
```

```
===== 13 passed in 1.00s =====
```

```
) manel@debian:~/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_block_2
```

CONCLUSION

Ce projet m'a permis de construire un backend complet sans utiliser de framework, en comprenant chaque étape du traitement d'une requête HTTP.

Même si ce type d'architecture n'est pas utilisé en production, il permet une meilleure compréhension des outils modernes comme Flask ou Django.

En le revisitant aujourd'hui, je peux identifier des améliorations possibles en termes de structure, sécurité et optimisation, ce qui me montre mon évolution en développement backend.

MERCI

Manel Ocana